

AWS & GitHub

프로젝트 초기 세팅 가이드

날씨 기반 생활 추천 서비스
Vue.js + FastAPI + AWS Serverless Architecture

문서 버전 1.0 | 2026년 3월 26일 | 팀원 5명 협업 기준

목차

목차	2
1. 문서 개요	3
2. GitHub 레포지토리 구성	4
2.1 모노레포 폴더 구조	4
2.2 브랜치 전략	4
2.3 보안 설정	5
3. AWS 초기 세팅 및 보안	6
3.1 루트 계정 보호	6
3.2 팀원별 IAM 계정 발급	6
3.3 예산 알람 설정 (AWS Budgets)	6
3.4 프리 티어 한도 참고	7
3.5 추가 초기 설정	7
CloudTrail 활성화	7
Service Quotas 확인	7
4. GitHub Actions CI/CD 설정	8
4.1 인증 방식: OIDC 연동 (필수)	8
OIDC 설정 순서	8
4.2 환경 분리 (Development / Production)	8
4.3 프론트엔드 (Vue.js) 배포 흐름	8
4.4 백엔드 (FastAPI + Mangum) 배포 흐름	9
4.5 데이터 파이프라인 배포 흐름	9
5. 배포 도구 선택: AWS SAM	10
6. 초기 세팅 체크리스트	11
6.1 AWS 초기 세팅	11
6.2 GitHub 세팅	11

1. 문서 개요

본 문서는 날씨 기반 생활 추천 서비스 프로젝트의 AWS 클라우드 환경 초기 설정과 GitHub 레포지토리 구성, CI/CD 파이프라인 구축을 포괄하는 세팅 가이드입니다.

항목	내용
프로젝트 구조	Vue.js 프론트엔드 + FastAPI 백엔드 + AWS Serverless
팀 규모	5명 (프론트엔드, 백엔드, 데이터, 인프라, PM)
주요 AWS 서비스	S3, CloudFront, Lambda, Lambda Function URL (API 엔드포인트), DynamoDB, Bedrock, Glue, Athena, EventBridge
권장 리전	ap-northeast-2 (서울)
배포 도구	GitHub Actions + AWS SAM

⚠ 시작 전 필수 확인 사항

모든 팀원이 동일한 AWS 리전(ap-northeast-2)을 사용하는지 확인하세요.
리전이 다르면 서비스 간 연동 오류와 불필요한 데이터 전송 비용이 발생합니다.

2. GitHub 레포지토리 구성

2.1 모노레포 폴더 구조

5명의 팀원이 효율적으로 협업하기 위해 모노레포(Monorepo) 방식으로 프론트엔드, 백엔드, 데이터 파이프라인, 인프라 코드를 하나의 레포에서 관리합니다.

```
project-root/
├── .github/workflows/      # GitHub Actions CI/CD 스크립트
│   ├── deploy-frontend.yml
│   ├── deploy-backend.yml
│   └── deploy-data-pipeline.yml
├── frontend/              # Vue.js 화면 코드
├── backend/                # FastAPI + Mangum 비즈니스 로직
├── data_pipeline/         # EventBridge + Lambda ETL 스크립트
├── infra/                  # IaC (SAM template.yaml 등)
├── .gitignore
└── README.md
```

☑ infra/ 폴더의 역할

SAM template.yaml, CloudFormation 템플릿 등 인프라 정의 파일을 코드로 버전 관리합니다. 팀원 간 환경 차이 문제를 예방하고, 인프라 변경 이력을 추적할 수 있습니다.

2.2 브랜치 전략

5명 협업 시 코드 충돌과 배포 사고를 방지하기 위해 브랜치 전략을 명확히 정해야 합니다.

브랜치	용도	배포 대상
main	운영 환경 배포용 (안정 버전만 병합)	Production
develop	통합 테스트용 (기능 병합 후 검증)	Development
feature/*	개별 기능 개발용	배포 안 함

⚠ 필수 규칙

feature 브랜치에서 develop으로의 병합은 반드시 PR(Pull Request) + 최소 1명 코드 리뷰 후 승인.
develop에서 main으로의 병합은 전체 테스트 통과 후에만 진행.
main 브랜치에 직접 Push는 절대 금지 (Branch Protection Rule 설정).

2.3 보안 설정

민감 정보가 GitHub에 유출되면 AWS 계정이 탈취될 수 있으므로 다층 방어를 설정합니다.

1. **.gitignore** 설정: .env, *.pem, node_modules/, __pycache__/, .aws/ 등을 반드시 추가
2. **git-secrets** 설치: pre-commit hook 으로 AWS Key 패턴을 감지하여 커밋 자체를 차단
3. **GitHub Secrets 활용**: 환경 변수는 GitHub Repository Settings → Secrets and variables 에 등록

3. AWS 초기 세팅 및 보안

클라우드 환경에서는 보안과 예산 관리가 가장 중요합니다. 계정 발급 직후, 개발 시작 전에 아래 세팅을 모두 완료하세요.

3.1 루트 계정 보호

1. AWS 콘솔 로그인 → IAM → 보안 자격 증명 → MFA(다중 인증) 활성화
2. 인증 앱(Google Authenticator 등)으로 QR 코드 스캔 후 등록 완료
3. 루트 계정의 Access Key 는 절대 발급하지 않음 (일상 작업에 사용 금지)

⚠ 루트 계정 보호가 중요한 이유

루트 계정은 모든 AWS 서비스에 무제한 접근이 가능합니다.

MFA 없이 탈취되면 비트코인 채굴, 대량 서버 생성 등으로 수천만 원의 요금이 발생할 수 있습니다.

3.2 팀원별 IAM 계정 발급

각 팀원에게 역할에 필요한 최소한의 권한만 부여합니다 (최소 권한 원칙).

역할	필요 AWS 권한	비고
프론트엔드	S3, CloudFront	정적 파일 업로드 및 CDN 관리
백엔드	Lambda, DynamoDB, Bedrock	API, AI 추천 및 데이터 저장소 관리
데이터	Lambda, EventBridge, S3, Glue, Athena, Bedrock	데이터 수집/배치 추론/분석 파이프라인
인프라	IAM, CloudFormation, SAM, CloudWatch	전체 인프라 및 모니터링 관리

✅ IAM Identity Center 고려

IAM 사용자에게 장기 Access Key를 발급하는 대신, IAM Identity Center(SSO)를 사용하면 팀원들이 콘솔과 CLI 모두 임시 자격 증명으로 접근하게 되어 보안이 훨씬 강화됩니다.

3.3 예산 알람 설정 (AWS Budgets)

계정 발급 직후 가장 먼저 설정해야 할 항목입니다. 예기치 않은 요금 폭탄을 방지합니다.

1. AWS Budgets 콘솔 → 예산 생성 → 월별 목표 금액 설정
2. 알람 기준값 설정: 예산의 50%, 80%, 100% 도달 시 이메일 알람
3. 수신자에 모든 팀원의 이메일 등록 (한 사람만 받으면 알람이 묵힘)

3.4 프리 티어 한도 참고

프로젝트에서 사용하는 주요 서비스의 무료 사용 한도입니다. 초과 시 과금이 발생하므로 팀원 전원이 숙지해야 합니다.

서비스	프리 티어 한도 (월간)	비고
Lambda	요청 100만 회 + 40만 GB-초 컴퓨팅	12개월 무기한
DynamoDB	저장 25GB + 읽기/쓰기 각 25단위	무기한
S3	저장 5GB + GET 2만 회 + PUT 2천 회	12개월 한정
CloudFront	데이터 전송 1TB + 요청 1,000만 회	무기한
Bedrock	입출력 토큰 기반 과금	프리티어 없음 – 주의
Glue	ETL 작업 DPU-시간 기반 과금	프리티어 없음 – 주의
Athena	스캔된 데이터 TB당 \$5	프리티어 없음 – 주의

⚠ Bedrock, Glue, Athena는 프리 티어가 없습니다

Bedrock, Glue, Athena는 사용하는 순간부터 과금됩니다.

데이터를 Parquet 포맷으로 변환하면 Athena 스캔 비용을 최대 95%까지 절감할 수 있습니다.

3.5 추가 초기 설정

CloudTrail 활성화

누가 어떤 AWS 작업을 했는지 추적하는 감사 로그입니다. 5명이 함께 쓸 때 문제 발생 시 원인 파악에 필수적이며, 기본 90일 이벤트 기록은 무료입니다.

Service Quotas 확인

신규 AWS 계정은 Lambda 동시 실행 수, Bedrock 모델 접근 권한 등이 기본값으로 낮게 설정되어 있습니다. 프로젝트 초기에 필요한 쿼터를 확인하고 미리 상향 요청해주세요.

4. GitHub Actions CI/CD 설정

개발자가 코드를 GitHub에 Push하면 자동으로 AWS에 배포되는 파이프라인을 구축합니다.

4.1 인증 방식: OIDC 연동 (필수)

장기 Access Key를 GitHub Secrets에 저장하는 방식은 보안상 위험합니다. OIDC(OpenID Connect)를 사용하면 배포 실행 시에만 짧은 시간 유효한 임시 자격 증명을 받아 안전하게 통신합니다.

OIDC 설정 순서

1. AWS IAM → Identity Providers → Add Provider → OpenID Connect 선택
2. Provider URL: <https://token.actions.githubusercontent.com>
3. Audience: sts.amazonaws.com
4. IAM Role 생성 → Trust Policy 에 특정 레포지토리/브랜치만 허용하는 조건 추가
5. GitHub Actions 워크플로에서 [aws-actions/configure-aws-credentials](#) 액션으로 role-to-assume 지정

⚠️ OIDC Trust Policy 설정 시 주의

Trust Policy에 레포 조건을 빠뜨리면 누구나 해당 Role을 사용할 수 있습니다.

반드시 `repo:<org>/<repo>:ref:refs/heads/<branch>` 형식으로 제한하세요.

4.2 환경 분리 (Development / Production)

모든 Push가 운영 환경에 직접 배포되면 위험합니다. 반드시 개발 환경과 운영 환경을 분리하세요.

환경	트리거 브랜치	배포 대상 리소스	S3 버킷 예시
Development	develop	별도 Lambda, S3, CloudFront	myapp-dev-frontend
Production	main	운영용 Lambda, S3, CloudFront	myapp-prod-frontend

i Path Filter 설정 필수

모노레포에서는 frontend/ 변경 시 프론트 파이프라인만, backend/ 변경 시 백엔드 파이프라인만 실행 되도록

GitHub Actions YAML에 `paths: ['frontend/**']` 필터를 반드시 추가하세요.

이 설정이 없으면 한 줄 수정에도 전체 배포가 실행되어 비효율적입니다.

4.3 프론트엔드 (Vue.js) 배포 흐름

1. 트리거: frontend/ 폴더에 코드 Push (develop 또는 main 브랜치)
2. 빌드: npm ci → npm run lint → npm run build 로 정적 파일 생성
3. 업로드: aws s3 sync dist/ s3://<bucket-name> --delete
4. 캐시 무효화: aws cloudfront create-invalidation --distribution-id <ID> --paths "/*"

⚠ CloudFront 캐시 무효화 주의사항

S3에 파일이 올라가도 CloudFront 캐시가 남아있으면 사용자에게 구 버전이 보입니다.

월 1,000건까지 무료이며, 빈번한 배포 시 /* 와일드카드 대신 변경된 파일 경로만 지정하면 비용을 절감할 수 있습니다.

4.4 백엔드 (FastAPI + Mangum) 배포 흐름

1. 트리거: backend/ 폴더에 코드 Push
2. 테스트: pip install → pytest 실행 (실패 시 배포 중단)
3. 패키징: FastAPI + Mangum + 의존성을 Lambda 배포 패키지로 묶음
4. 배포: AWS SAM 으로 sam build → sam deploy 실행 (Lambda 함수 업데이트)

⚠ Lambda 패키징 용량 제한

Lambda ZIP 배포는 압축 해제 시 250MB 제한이 있습니다.

의존성이 많으면 Lambda Layer로 분리하거나, 컨테이너 이미지 배포(최대 10GB)를 사용하세요.
프로젝트 초기에 패키징 방식을 미리 결정해두는 것을 권장합니다.

4.5 데이터 파이프라인 배포 흐름

data_pipeline/ 폴더의 EventBridge + Lambda + Glue 관련 리소스도 CI/CD에 포함해야 합니다.

- EventBridge 규칙, Lambda 함수, Glue Job 을 SAM template.yaml 에 정의
- data_pipeline/ 변경 시 deploy-data-pipeline.yml 워크플로가 실행되도록 path filter 설정
- Athena 테이블 생성 DDL 은 infra/ 폴더에 SQL 파일로 버전 관리

5. 배포 도구 선택: AWS SAM

본 프로젝트에서는 AWS 네이티브 도구인 AWS SAM(Serverless Application Model)을 표준 배포 도구로 사용합니다.

항목	AWS SAM	Serverless Framework
장점	AWS 공식 도구, CloudFormation 완전 호환	풍부한 플러그인 생태계
단점	플러그인 제한적	별도 계정/설정 필요
추천 상황	AWS 전용 프로젝트 (본 프로젝트)	멀티 클라우드 프로젝트

⚠ 배포 도구는 반드시 하나로 통일

팀원들이 각각 다른 도구로 작업하면 설정 파일 충돌이 발생합니다.
프로젝트 시작 전에 SAM으로 확정하고, 모든 팀원이 SAM CLI를 설치하세요.

6. 초기 세팅 체크리스트

아래 항목을 순서대로 완료하면 개발을 시작할 준비가 됩니다.

6.1 AWS 초기 세팅

- ☐ 루트 계정 MFA 활성화
- ☐ 리전 통일 확인 (ap-northeast-2)
- ☐ AWS Budgets 예산 알람 설정 (50% / 80% / 100%)
- ☐ CloudTrail 활성화 확인
- ☐ 팀원별 IAM 계정 발급 + 최소 권한 적용
- ☐ Service Quotas 확인 및 상향 요청

6.2 GitHub 세팅

- ☐ 모노레포 생성 + 폴더 구조 세팅
- ☐ .gitignore 및 git-secrets 설정
- ☐ Branch Protection Rule 적용 (main, develop)
- ☐ OIDC 연동 완료 (AWS Identity Provider + IAM Role)
- ☐ GitHub Actions 워크플로 작성 (path filter 포함)
- ☐ 환경 분리 확인 (dev/prod S3 버킷, Lambda 함수 별도 생성)